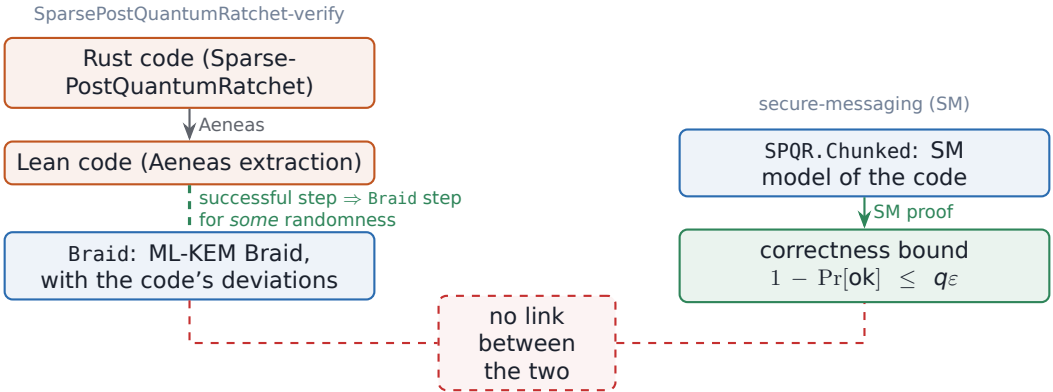
Transferring secure-messaging's SCKA correctness bound to the extracted SPQR code

SpqrBridge

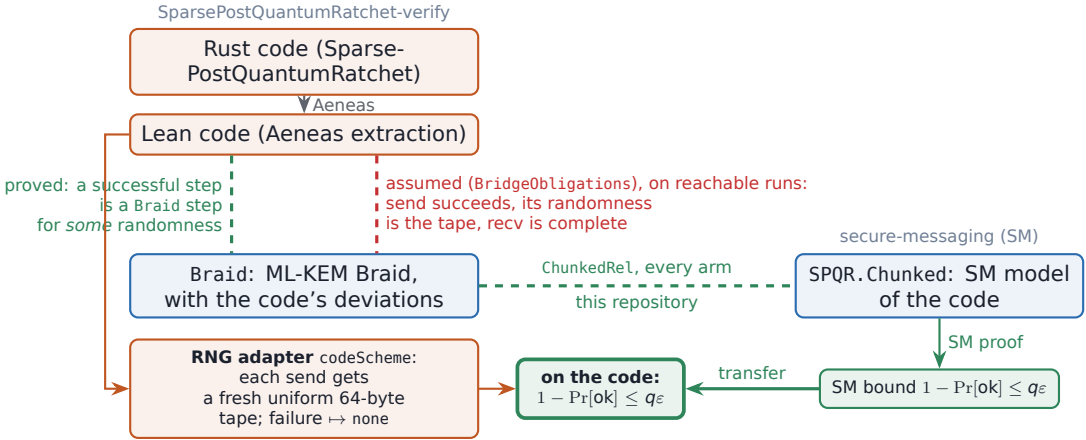
November 2026



Two proofs about SPQR, one gap



A conditional bridge across the gap



The bound needs the dashed arrow, not the proved one: "some randomness" says nothing about its distribution. Issues #785, #786 and #787 own the dashed arrow.

The result

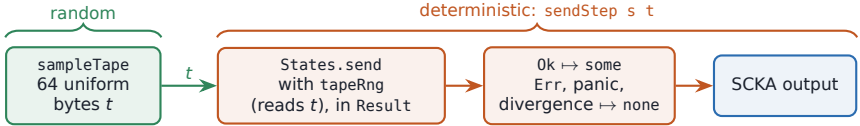
Assume the four facts about the code in `BridgeObligations`, which are not proved yet. The send and receive facts cover reachable runs only. Let `adv` make at most q send queries, counted over both parties, with $q + 2 < 2^{64}$.
Then the probability that the correctness game fails on the code is at most $q\varepsilon$.

```
theorem code_correctness_error_le (hB : BridgeObligations)
    (sampleInitKey : ProbComp InitKey)
    (adv : SCKACorrectnessAdversary Message) (q : ℕ)
    (hq : SendQueryBound adv q) (hq64 : q + 2 < 2 ^ 64) :
    1 - Pr{let x ← correctnessExp (codeScheme sampleInitKey) adv}
          [x = true] ≤
      (q : ℝ≥0∞) * codeKem.correctnessError ProbCompRuntime.probComp
```

- `codeScheme`: the code as an SCKA scheme, with fresh uniform bytes for each send (slide 5).
- ε: the correctness error of `codeKem`, the KEM of the code's ML-KEM models. It has no numerical bound yet.
- The adversary delivers unmodified recorded messages, in any order and with replay. No injected messages, no liveness.
- No `sorry`. Axioms beyond Lean's three: HKDF, HMAC, one libcrux workaround.

Where the code starts running in `ProbComp`

A send of the code: `codeSendA s = do let t ← sampleTape; pure (sendStep s t)`
Only `sampleTape` is random. All that follows is a plain function of the tape *t*.



spqr-verify

Open, #785 `send_total`: a reachable send returns `Ok`.

Open, #787 `send_refines_tape`: the matching `Braid` step reads the same bytes of *t*.

this repository

`codeSend_eq_some_iff`: `some` iff `Ok`.

`dropRng_send_prefix`: a send reads at most 64 bytes of *t*.

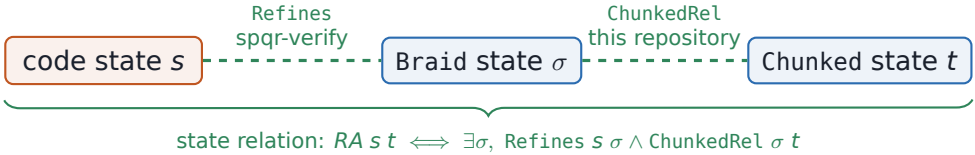
`send_chunked`: on the same *t*, the sends give related results.

secure-messaging (SM)

`SPQR.Chunked.send` samples its own bytes. With `codeParams`, it also uses `sampleTape`.

- Assumed, not in Lean: the real RNG acts like `sampleTape` on each Rust send (slide 11).
- Divergence maps to `none`, because a `ProbComp` must return a value. `send_total` rules it out on reachable sends.

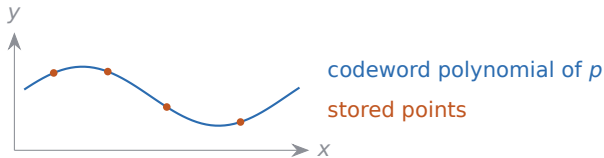
The chain of relations



- *RA*: a code state and an SM state are related through `Braid`. The code state refines some `Braid` state σ (spqr-verify), and σ is related to the SM state (`ChunkedRel`). Both parties use it.
- `ChunkedRel`: the same of the eleven states, equal epochs and authenticators, byte fields as typed values, equal decoders, and each code encoder's `u32` counter is SM's counter modulo 2^{32}. The header that `Refines` leaves free is taken from the SM state.
- *RM*: `Braid`'s message map `absMsg`, followed by the chunk map $c \mapsto (c.\texttt{index}, c.\texttt{data})$ into SM.
- `send_chunked` and `receive_chunked` cover every arm of both machines.

The Rust decoder meets SM's decoder law

SM's machines are generic in a `StreamDecoder`. SM's own decoder decodes a set of chunks. The Rust decoder skips points as they arrive, so its state depends on the order. SM's proofs need one law, `Lawful`: adding a chunk of p to a decoder that is honest for p keeps it honest, and it then decodes nothing or p.

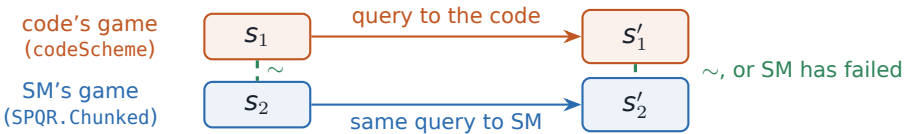


- The payload p gives 16 slot polynomials over $\mathrm{GF}(2^{16})$, each of degree $< K$.
- Honest state: every stored point lies on the slot polynomial of p. An honest chunk adds a point of the same polynomial, or is skipped.
- K points with distinct x fix a polynomial of degree $< K$, so a decoded value is p.
- `polyStreamDecoder_lawful`: one theorem for the four streams, $K \in \{3, 36, 30, 5\}$.
- The law is safety only: a decoder that never decodes meets it too.

Moving the bound to the code

SM's bound comes from `correctness_error_le_of_potential`, over an invariant Inv_2 and a potential V_2 on its game states. The bridge builds V_1 from V_2. Here $s_1 \sim s_2$ relates party states by *RA* and recorded messages by *RM*, with equal key tables, counters and flag.

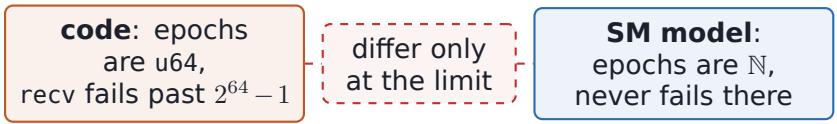
$$V_1(s_1) = \begin{cases} 1 & \text{if } s_1 \text{ has failed} \\ \min\big(1,\ \inf\{\, V_2(s_2) \mid s_1 \sim s_2,\ Inv_2\ s_2 \,\}\big) & \text{otherwise} \end{cases}$$



- One step, for *every* related s_2 with $Inv_2\ s_2$: $\mathbb{E}[V_1(s'_1)] \le \mathbb{E}[V_2(s'_2)] \le V_2(s_2) + c$, with $c = \varepsilon$ for a send and $c = 0$ otherwise. The first $\le$ holds because the states are related again, or SM failed and $V_2 \ge 1 \ge V_1$ (slide 10).
- The bound survives the infimum and the cap. No related s_2 gives $V_1 = 1$. No witness is chosen, and the games need not show the adversary equal values (`correctness_error_le_of_budgeted_transfer`).

Why 64-bit epochs never overflow

An adversary with fewer than $2^{64} - 2$ sends cannot push an epoch to the overflow point.



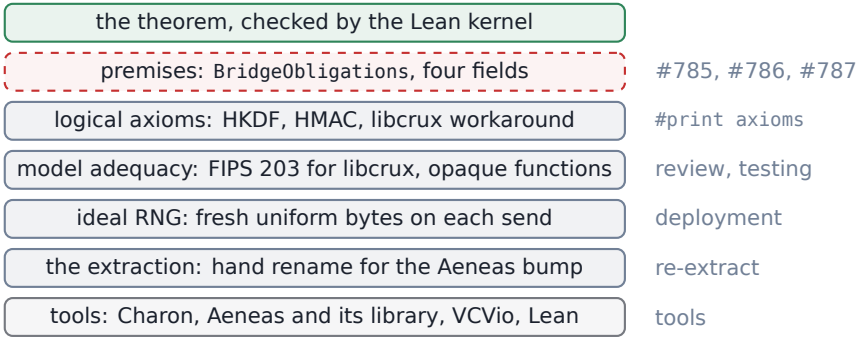
1. At the limit the two machines differ, so they cannot be related there.
2. Every epoch is at most the number of successful sends plus one (`codeGameInv_epochs`).
3. So the step obligations are needed only below a cap $B = 2^{64} - 2$ on the sends: fewer than B for a send, at most B for a receive, since deliveries go on after the last send. With $q + 2 < 2^{64}$, the adversary's q sends stay below B.

Derived transfer obligations

init	Every pair of initial states that the code can start in is related to a pair that SM can start in.
send	The two sends are coupled on one 64-byte sample. For every value of it, their results are related.
recv	If SM accepts a recorded message, the code accepts it too, with related results. If SM refuses, nothing is required.

- A send reads 0, 32 or 64 bytes of the sample: key generation reads 64, encapsulation 32. For these two, `codeParams` builds SM's operations from the code's own primitives, so both read the same bytes.
- The other sends read none. The code ignores its bytes, and both results are fixed and related.
- Why nothing is required when SM refuses: SM's game has failed already, and the bound counts that failure.

What the theorem trusts



- `#print axioms` audits only the axioms layer. The models, the opaque functions (SHA3, HMAC, libcrux sizes) and the RNG do not appear in it.
- A wrong model does not always break a premise: both sides of the bridge can agree on it. So the theorem is about the models, and their adequacy is a separate claim.
- ε is built from these models, so a bad primitive makes the bound weak, not false.

Status and next steps

piece	where	status
code refines `Braid`	spqr-verify	proved, one direction
generic stream decoders	SM	proved, merged
`Braid` related to `SPQR.Chunked`	this repository	proved, every arm
Rust decoder is lawful	this repository	proved
transfer of the bound	this repository	proved, for $q + 2 < 2^{64}$
premise `mlkem_sizes`	spqr-verify, #786	open
premises `send_total`, `recv_complete`	spqr-verify, #785	open
premise `send_refines_tape`	spqr-verify, #787	open
a number for ε	#788, #799, a model link	open

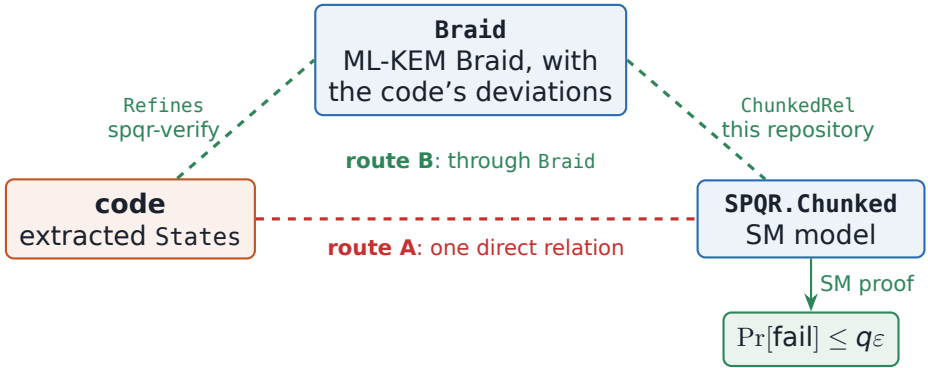
spqr-verify: SparsePostQuantumRatchet-verify. SM: secure-messaging.

Next

- Build the exact `BridgeObligations` record from upstream theorems (#785, #786, #787). Closing an issue is not enough if its statement differs.
- Bound ε: the #788 workaround, the `sampleNTT` cap (#799), and a proof that the two FIPS 203 models agree (README).

Two routes from SM's bound to the code

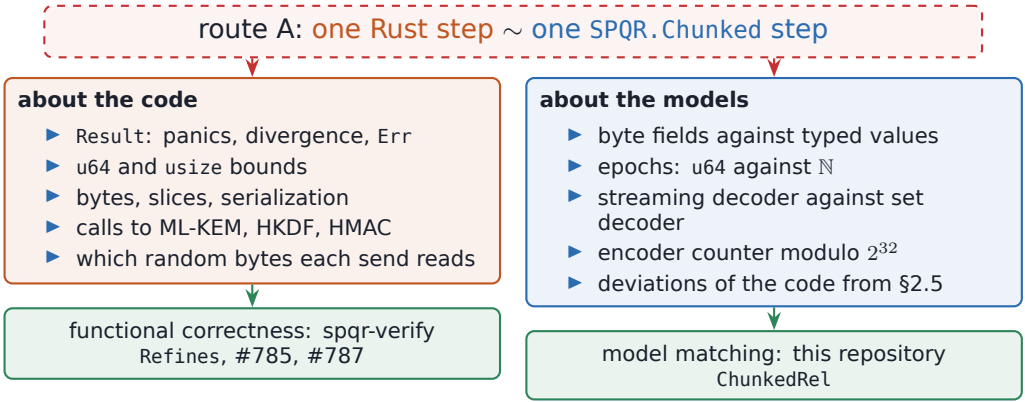
SM proves its bound for the game over `SPQR.Chunked`. To move it, relate each state of the code's game to a state of SM's game, step by step.



- Both routes must give one relation *RA* between code states and SM states. Sends are coupled, and receives keep it until SM fails.
- Route B builds it as a composition through `Braid` (slide 6).

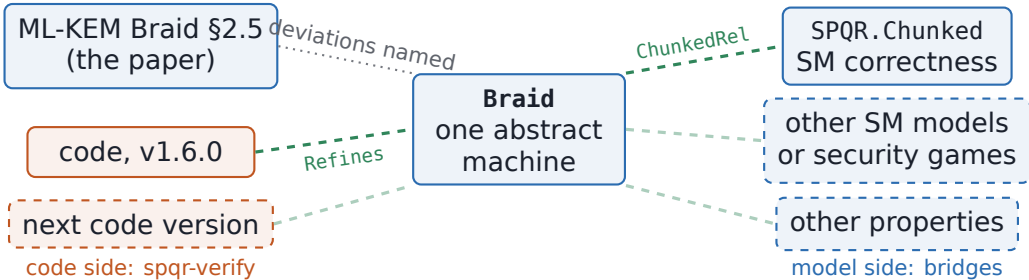
What a direct relation would have to prove

A step of route A relates one Rust step to one SM step. It mixes two kinds of facts.



Route A needs the left column too. It can reuse upstream lemmas, but it states each step against SM's types. Route B states these facts once, against `Braid`. Totality and the tape facts are still premises there (#785, #787).

Why `Braid` in the middle



- Each repository proves what it owns: facts about the Rust code next to the extraction, facts about SM's model next to SM.
- If `Braid`'s interface stays stable, a change on one side redoes one relation only: a new code version redoes `Refines`, a new model needs a relation from `Braid`.
- Security games are future work. They see leaked randomness, so a fresh tape per send and one threaded tape are no longer interchangeable.
- `Braid` reads like the paper, with each deviation named.
- The cost: one more model to maintain. `ChunkedRel` still needs two code facts: the Rust decoder is lawful, and the encoder counter wraps.